Writer Monad

Writer · Compute totals while accumulating audit log · Build with Waqar

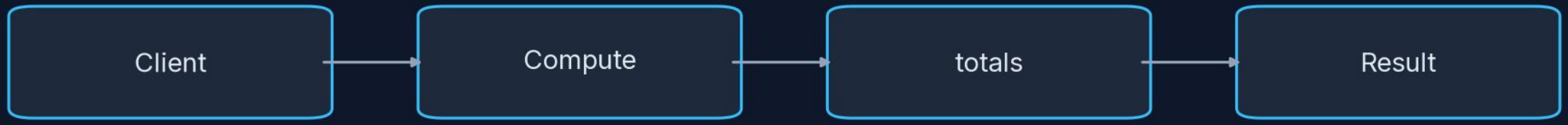
Today's Hook

- Problem: Compute totals while accumulating audit log
- Pattern: Writer — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Compute totals while accumulating audit log
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Writer: Compute totals while accumulating audit log

Real-World Scenario

- Running example: Compute totals while accumulating audit log
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.writer;

import com.buildwithwaqar.patterns.functional.writer.Writer;
import com.buildwithwaqar.patterns.functional.monoid.Monoids;
/** Problem: compute a total while accumulating an audit trail. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Writer problem: audited total ===");
    var m = Monoids.STRING;
    var w = Writer.<String, Integer>pure(m, 0)
      .flatMap(m, n -> new Writer<>("add10;", n + 10))
      .flatMap(m, n -> new Writer<>("add5;", n + 5));
    System.out.println("value=" + w.value() + " log=" + w.log());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.writer;

import com.buildwithwaqar.patterns.functional.monoid.Monoid;

import java.util.function.Function;

public record Writer<W, A>(W log, A value) {
    public <B> Writer<W, B> map(Function<? super A, ? extends B> f) {
        return new Writer<>(log, f.apply(value));
    }

    public <B> Writer<W, B> flatMap(Monoid<W> monoid, Function<? super A, Writer<W, B>> f) {
        var next = f.apply(value);
        return new Writer<>(monoid.combine(log, next.log), next.value);
    }

    public static <W, A> Writer<W, A> pure(Monoid<W> monoid, A a) {
        return new Writer<>(monoid.empty(), a);
    }

    public static <W> Writer<W, Void> tell(W w) { return new Writer<>(w, null); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.writer;

import com.buildwithwaqar.patterns.functional.writer.Writer;
import com.buildwithwaqar.patterns.functional.monoid.Monoids;
/** Problem: compute a total while accumulating an audit trail. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Writer problem: audited total ===");
    var m = Monoids.STRING;
    var w = Writer.<String, Integer>pure(m, 0)
      .flatMap(m, n -> new Writer<>("add10;", n + 10))
      .flatMap(m, n -> new Writer<>("add5;", n + 5));
    System.out.println("value=" + w.value() + " log=" + w.log());
  }
}
```

Walkthrough Output

```
=== Writer problem: audited total ===
value=15 log=add10;add5;
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Writer.

Related Patterns

- Contrast Writer with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Writer → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.